

2.4. Sumas Hipergeométricas*

A lo largo de este capítulo hemos demostrado, mediante diferentes técnicas combinatorias, igualdades de la forma $A = B$, donde A es una suma que involucra términos como coeficientes binomiales, y B es una fórmula *cerrada* o una nueva suma. Un ejemplo es la suma de los coeficientes binomiales:

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad n \geq 0.$$

Otro ejemplo es la suma de los recíprocos de los coeficientes binomiales:

$$\sum_{k=0}^n \frac{1}{\binom{n}{k}} = \frac{n+1}{2^n} \sum_{k=0}^n \frac{2^k}{k+1}.$$

Este tipo de problemas puede abordarse de forma computacional mediante algunos algoritmos, como el **algoritmo de Gosper** [**Gosper**], el **algoritmo de Zeilberger** [**Zeil1**] y el **método WZ** [**WZZ**]. En particular, para sumas de la forma

$$\sum_k F(n, k) = C(n)$$

se tiene que:

1. Si $C(n)$ es desconocida, el objetivo de los algoritmos de Gosper y Zeilberger es encontrar $C(n)$ a partir de relaciones de recurrencia que satisfacen el término $F(n, k)$.
2. Si $C(n)$ es conocida, el objetivo del método WZ es demostrar que $\sum_k \frac{F(n, k)}{C(n)} = 1$, a partir de una recurrencia que satisface el término $\frac{F(n, k)}{C(n)}$.

Esta sección tiene como objetivo introducir brevemente, sin entrar en detalles formales, este tipo de técnicas (para más detalles, ver [**WZ**, **Koepf**]). El problema original de desarrollar algoritmos computacionales para simplificar sumas que involucren coeficientes binomiales fue propuesto por Donald Knuth en el Ejercicio 1.2.6.63 de su libro *The Art of Computer Programming, Volume 1: Fundamental Algorithms*.

Antes de presentar los algoritmos, necesitamos algunas definiciones.

Definición 2.7 Una sucesión $\{t_n\}_{n \geq 0}$ se denomina **hipergeométrica** si el cociente t_{n+1}/t_n es una función racional en n ; es decir, existen polinomios $p(n)$ y $q(n)$ tales que

$$\frac{t_{n+1}}{t_n} = \frac{p(n)}{q(n)}, \quad \text{para todo } n \geq 0.$$

A este cociente se le llama **término hipergeométrico**, y a la suma $\sum_{k \geq 0} t_k$ se le llama **suma hipergeométrica**.

Ejemplo 2.4 A partir de la definición, vemos que las sucesiones $n!$, 2^n y $\binom{n}{k}$ son términos hipergeométricos en n . De hecho, el coeficiente binomial es hipergeométrico en ambas variables. Esto puede comprobarse con los siguientes comandos en *Mathematica*[®]:

```
In[14]:= FunctionExpand[Binomial[n, k + 1]/Binomial[n, k]]
```

```
Out[14]=  $\frac{-k+n}{1+k}$ 
```

```
In[15]:= FunctionExpand[Binomial[n + 1, k]/Binomial[n, k]]
```

```
Out[15]=  $\frac{1+n}{1-k+n}$ 
```

Ejemplo 2.5 Las siguientes series de potencias son ejemplos de sumas hipergeométricas:

$$e^x = \sum_{n \geq 0} \frac{x^n}{n!}, \quad y \quad \sin x = \sum_{n \geq 0} (-1)^n \frac{x^{2n+1}}{(2n+1)!}.$$

En ambos casos, los términos de las series forman sucesiones hipergeométricas. Para la serie de e^x , el cociente entre términos consecutivos es

$$\frac{a_{n+1}}{a_n} = \frac{x^{n+1}/(n+1)!}{x^n/n!} = \frac{x}{n+1},$$

que es una función racional en n .

En la serie de $\sin x$, el término general es

$$a_n = (-1)^n \frac{x^{2n+1}}{(2n+1)!},$$

y el cociente entre términos consecutivos resulta ser

$$\frac{a_{n+1}}{a_n} = \frac{(-1)^{n+1} x^{2n+3}/(2n+3)!}{(-1)^n x^{2n+1}/(2n+1)!} = \frac{-x^2}{(2n+2)(2n+3)},$$

que también es racional en n .

Definición 2.8 Decimos que una función $f(n)$ está expresada en **forma cerrada** si, para todo n , $f(n)$ puede escribirse como una combinación lineal de una cantidad fija r (independiente de n) de términos hipergeométricos en n .

2.4.1. Algoritmo de Gosper

En los cursos de cálculo se estudia el *Teorema Fundamental del Cálculo*, que establece lo siguiente: si $f(x)$ es una función continua en un intervalo cerrado $[a, b]$, y $F(x)$ es una función tal que $F'(x) = f(x)$ en dicho intervalo, entonces

$$\int_a^b f(t) dt = F(b) - F(a).$$

La función F se denomina *antiderivada* de f . En muchos casos, es posible calcular integrales definidas utilizando tablas de antiderivadas. Cuando esto no es posible, existen diversos métodos y *trucos* algebraicos que pueden aplicarse. Además, el **algoritmo de Risch** permite decidir, de manera algorítmica, si una función admite una integral indefinida que pueda expresarse en términos de funciones elementales [**BookComAlgebra**].

En el contexto discreto, el problema análogo consiste en determinar cuándo una suma indefinida puede expresarse en forma cerrada. El **algoritmo de Gosper**, propuesto por R. W. Gosper en 1978 [**Gosper**], resuelve esta cuestión para sumas de términos hipergeométricos. En particular, responde a la siguiente pregunta:

¿Existe una anti-diferencia s_k para un término hipergeométrico t_k ?

Es decir, dado un término hipergeométrico t_k , se desea encontrar una sucesión hipergeométrica $\{s_k\}_{k \geq 0}$ tal que $t_k = \Delta s_k = s_{k+1} - s_k$. Una vez conocida una anti-diferencia s_k de t_k , el cálculo de una fórmula cerrada para la suma

$$\sum_{k=m}^n t_k$$

se simplifica de forma inmediata. Al reemplazar cada término t_k por su anti-diferencia, se obtiene la suma telescópica:

$$\sum_{k=m}^n t_k = (s_{m+1} - s_m) + (s_{m+2} - s_{m+1}) + \cdots + (s_{n+1} - s_n) = s_{n+1} - s_m.$$

Cuando dicha sucesión s_k existe, diremos que la suma $\sum_{k=m}^n t_k$ es una **suma del tipo Gosper**.

Supongamos ahora que s_n es una anti-diferencia de t_n , es decir, que satisface $s_{n+1} - s_n = t_n$. Consideremos entonces el cociente

$$\frac{s_n}{t_n} = \frac{s_n}{s_{n+1} - s_n} = \frac{1}{\frac{s_{n+1}}{s_n} - 1}.$$

Dado que esta expresión es racional en n , podemos suponer que $s_n = y(n)t_n$, donde $y(n)$ es una función racional en n . Por lo tanto, se obtienen las siguientes equivalencias:

$$\begin{aligned} s_{n+1} - s_n = t_n &\Leftrightarrow y(n+1)t_{n+1} - y(n)t_n = t_n \\ &\Leftrightarrow y(n+1)\frac{t_{n+1}}{t_n} - y(n) = 1 \\ &\Leftrightarrow y(n+1)r(n) - y(n) = 1, \end{aligned}$$

donde $r(n) = \frac{t_{n+1}}{t_n}$ es racional en n . Así, el problema de encontrar una anti-diferencia s_n de t_n se reduce a resolver la recurrencia de primer orden donde los coeficientes son funciones racionales:

$$r(n)y(n+1) - y(n) = 1. \quad (2.2)$$

Gosper demostró que esta recurrencia puede transformarse en otra de coeficientes polinomiales. En efecto, se puede suponer que

$$r(n) = \frac{a(n)c(n+1)}{b(n)c(n)},$$

donde $a(n)$, $b(n)$ y $c(n)$ son polinomios en n tales que $\text{mcd}(a(n), b(n+h)) = 1$ para todo entero positivo h [Koepf]. Al sustituir esta expresión en (2.2), se obtiene:

$$\left(\frac{a(n)c(n+1)}{b(n)c(n)} \right) y(n+1) - y(n) = 1,$$

que puede reescribirse como

$$\left(\frac{a(n-1)c(n)}{b(n-1)c(n-1)} \right) y(n) - y(n-1) = 1.$$

Despejando $y(n)$, se concluye que:

$$y(n) = (1 + y(n-1)) \cdot \frac{b(n-1)c(n-1)}{a(n-1)c(n)} = \frac{b(n-1)}{c(n)} x(n),$$

donde $x(n)$ es racional en n . Al sustituir esta expresión en (2.2), se obtienen las siguientes equivalencias:

$$\begin{aligned} r(n)y(n+1) - y(n) = 1 &\Leftrightarrow \frac{a(n)c(n+1)}{b(n)c(n)} \frac{b(n)}{c(n+1)} x(n+1) - \frac{b(n-1)}{c(n)} x(n) = 1 \\ &\Leftrightarrow \frac{a(n)}{c(n)} x(n+1) - \frac{b(n-1)}{c(n)} x(n) = 1 \\ &\Leftrightarrow a(n)x(n+1) - b(n-1)x(n) = c(n). \end{aligned}$$

Bajo estas condiciones, Gosper probó que $x(n)$ es un polinomio en n . Por lo tanto, el problema original de encontrar una anti-diferencia s_n de t_n se reduce a resolver la siguiente recurrencia de primer orden con coeficientes polinomiales:

$$a(n)x(n+1) - b(n-1)x(n) = c(n), \quad (2.3)$$

donde $a(n)$, $b(n)$ y $c(n)$ son polinomios en n . Para resolver este tipo de recurrencias existen métodos algorítmicos, descritos por ejemplo en [WZ]. En caso de que la recurrencia pueda resolverse, se concluye que

$$s_n = \frac{b(n-1)x(n)}{c(n)} \cdot t_n.$$

En la Figura 2.24 se resume el procedimiento descrito anteriormente, conocido como el **algoritmo de Gosper**.

Entrada Un término hipergeométrico t_n .

Salida Una anti-diferencia s_n (término hipergeométrico) de t_n .

- 1: Definimos el término $r(n) = \frac{t_{n+1}}{t_n} \in \mathbb{Q}(n)$.
- 2: Escribimos el término $r(n)$ como

$$r(n) = \frac{a(n)c(n+1)}{b(n)c(n)},$$

donde $a(n)$, $b(n)$ y $c(n)$ son polinomios en n y $\text{mcd}(a(n), b(n+h)) = 1$, para todo entero positivo h .

- 3: Resolvemos la recurrencia $a(n)x(n+1) - b(n-1)x(n) = c(n)$.
- 4: En caso que exista la solución, devuelva

$$s_n = \frac{b(n-1)x(n)}{c(n)} t_n.$$

Figura 2.24: Algoritmo de Gosper.

A continuación mostramos cómo utilizar el algoritmo de Gosper en *Mathematica*[®]. Para ello, se puede descargar² desde la página web del *Research Institute for Symbolic Computation* (RISC) el paquete **RISCergoSum**.

Una vez descargado, el paquete debe descomprimirse y guardarse en la carpeta local de paquetes de *Mathematica*[®].

Ejemplo 2.6 Evaluemos en forma cerrada la suma hipergeométrica

$$\sum_{k=0}^n \frac{k^4 4^k}{\binom{2k}{k}}.$$

Usando el comando `Gosper[t[k], {k, 0, n}]` del paquete **RISCergoSum**, donde $t[k]$ es el sumando hipergeométrico, podemos obtener el valor de la suma en caso de que sea del tipo Gosper.

```
In[16]:= << RISC'fastZeil'
```

Fast Zeilberger Package version 3.61
written by Peter Paule, Markus Schorn, and Axel Riese
Copyright Research Institute for Symbolic Computation (RISC),
Johannes Kepler University, Linz, Austria

```
In[17]:= Gosper[(k^4*4^k)/Binomial[2 k, k], {k, 0, n}]
```

²La clave para descomprimir los paquetes puede solicitarse al profesor Carsten Schneider a través del correo electrónico PCarsten.Schneider@risc.jku.at.

```
Out[17]= If 'n' is a natural number, then:
          4^k k^4
          {Sum[-----, {k, 0, n}] == -2
          Binomial[2 k, k]
          231
          + 2^{-1+2n}(1 + n)(3 - 22 n + 18 n^2 + 112 n^3 + 63 n^4)}
          693 Binomial[2 n, n]}
```

Además del valor de la suma, el algoritmo también permite obtener más información. Definamos

$$F(n, k) = \frac{k^4 4^k}{\binom{2k}{k}}.$$

El paquete encuentra una función $G(n, k)$ tal que

$$F(n, k) = \Delta_k(G(n, k)) = G(n, k+1) - G(n, k),$$

y el cociente

$$R(n, k) = \frac{G(n, k)}{F(n, k)}$$

resulta ser una función racional en ambas variables. A esta función se le conoce como el **certificado** de la identidad.

Para este ejemplo, tenemos:

```
In[18]:= F[n_, k_] := (k^4*4^k)/Binomial[2 k, k]
```

```
In[19]:= Gosper[F[n, k], k]
```

```
Out[19]= {----- ==
          4^k k^4
          Binomial[2 k, k]
          4^k(-1 + 2 k)(-6 + 26 k + 60 k^2 - 140 k^3 + 63 k^4)
          Delta_k[-----]}
          693 Binomial[2 k, k]
```

Definimos entonces la función $G(n, k)$:

```
In[20]:= G[n_, k_] := 4^k(-1 + 2 k)(-6 + 26 k + 60 k^2 - 140 k^3 + 63 k^4)
          693 Binomial[2 k, k]
```

```
In[21]:= FullSimplify[F[n, k] - (G[n, k + 1] - G[n, k])]
```

```
Out[21]= 0
```

Y calculamos el certificado:

```
In[22]:= G[n, k]/F[n, k]
```

```
Out[22]= (-1 + 2 k)(-6 + 26 k + 60 k^2 - 140 k^3 + 63 k^4)
          693 k^4
```

Esto demuestra que:

$$\frac{k^4 4^k}{\binom{2k}{k}} = F(n, k) = G(n, k+1) - G(n, k),$$

con

$$R(n, k) = \frac{G(n, k)}{F(n, k)} = \frac{(2k-1)(63k^4 - 140k^3 + 60k^2 + 26k - 6)}{693k^4}.$$

Por lo tanto,

$$\sum_{k=0}^n \frac{k^4 4^k}{\binom{2k}{k}} = G(n, n+1) - G(n, 0).$$

Obsérvese que el certificado obtenido mediante el algoritmo de Gosper constituye, en esencia, una demostración de la identidad hipergeométrica. En efecto, si se conoce el certificado $R(n, k)$, una función racional en n y k , entonces basta con definir

$$G(n, k) = F(n, k)R(n, k),$$

y verificar que

$$F(n, k) = G(n, k+1) - G(n, k).$$

A continuación, se procede a sumar ambos lados sobre k , desde el valor inicial hasta n :

$$\sum_{k=0}^n F(n, k) = \sum_{k=0}^n (G(n, k+1) - G(n, k)) = G(n, n+1) - G(n, 0).$$

Esta última diferencia corresponde al valor cerrado de la suma.

Lo anterior muestra que el algoritmo de Gosper es un ejemplo claro de lo que se conoce como una **demostración computacional**, en la que la veracidad de la identidad se reduce a una verificación algebraica explícita.

Ejemplo 2.7 Para todo entero positivo n se tiene que

$$\sum_{r=0}^n (-1)^r \binom{n}{r} \binom{2n-2r}{n-1} = 0.$$

Sea $F(n, r)$ el sumando de esta expresión, es decir

$$F(n, r) = (-1)^r \binom{n}{r} \binom{2n-2r}{n-1}.$$

Aplicando el algoritmo de Gosper implementado en *Mathematica*® obtenemos que

$$F(n, r) = G(n, r+1) - G(n, r),$$

con el certificado

$$R(n, r) = \frac{G(n, r)}{F(n, r)} = \frac{2r(-1-2n+2r)}{n+n^2}.$$

Por lo tanto,

$$\sum_{r=0}^n F(n, r) = G(n, n+1) - G(n, 0) = 0.$$

Quedando demostrado.

El siguiente ejemplo muestra que no todas las sumas hipergeométricas son del tipo Gosper.

Ejemplo 2.8 Consideremos la suma de coeficientes binomiales

$$\sum_{k=0}^n \binom{n}{k}.$$

Esta suma no es del tipo Gosper, como muestra la ejecución del comando correspondiente:

```
In[23]:= F[n_, k_] := Binomial[n, k]
```

```
In[24]:= Gosper[F[n, k], k]
```

```
Out[24]= { }
```

La salida vacía indica que la ecuación $s_{k+1} - s_k = \binom{n}{k}$ no admite una solución hipergeométrica. Es decir, no existe una anti-diferencia hipergeométrica para $t_k = \binom{n}{k}$. En cambio, si consideramos la suma alternada

$$\sum_{k=0}^n (-1)^k \binom{n}{k},$$

esta sí es del tipo Gosper. En este caso, el certificado asociado es

$$R(n, k) = -\frac{k}{n}.$$

2.4.2. Algoritmo de Zeilberger

Considérese la suma

$$f(n) := \sum_k F(n, k),$$

donde $F(n, k)$ es un término hipergeométrico en las variables n y k . Es importante destacar que la suma se extiende sobre todos los enteros k , salvo que se indique lo contrario.

El objetivo es encontrar una relación de recurrencia para la función $f(n)$. El punto de partida consiste en analizar si es posible obtener una relación de recurrencia para el sumando $F(n, k)$. Como ya se ha visto, no todas las sumas son del tipo Gosper. Por ejemplo, la suma

parcial $\sum_{k=0}^m \binom{n}{k}$ (con $m \leq n$) no admite una anti-diferencia hipergeométrica, es decir, no siempre se puede encontrar un término $G(n, k)$ tal que

$$F(n, k) = G(n, k+1) - G(n, k).$$

No obstante, en algunos casos es posible hallar un término $G(n, k)$ tal que se cumpla la siguiente identidad:

$$F(n+1, k) - F(n, k) = G(n, k+1) - G(n, k). \quad (2.4)$$

Si se satisface esta relación, se puede deducir que la suma $f(n)$ es constante. En efecto, al sumar ambos lados de (2.4) sobre todos los valores de k , se obtiene:

$$f(n+1) - f(n) = \sum_k (F(n+1, k) - F(n, k)) = \sum_k (G(n, k+1) - G(n, k)) = 0.$$

De esta forma, se concluye que $f(n) = f(0)$ para todo n . Este principio constituye la base del **algoritmo de Zeilberger**, que permite automatizar la búsqueda de relaciones de recurrencia para sumas hipergeométricas, incluso cuando no se dispone de una anti-diferencia directa como en el caso de Gosper.

Consideremos los operadores en diferencias definidos por

$$N(g(n, k)) := g(n+1, k) \quad \text{y} \quad K(g(n, k)) := g(n, k+1).$$

Con esta notación, la ecuación (2.4) puede reescribirse como

$$(N-1)F = (K-1)G.$$

El **algoritmo de Zeilberger** garantiza que, dado un término hipergeométrico $F(n, k)$, existe un operador en diferencias de la forma

$$p(n, N) = a_0(n) + a_1(n)N + a_2(n)N^2 + \cdots + a_J(n)N^J,$$

donde los coeficientes $a_i(n)$ son polinomios en n , tal que

$$p(n, N)F(n, k) = (K-1)G(n, k), \quad (2.5)$$

con la propiedad adicional de que el cociente $G(n, k)/F(n, k)$ es una función racional en n y k .

Esta importante observación fue demostrada por primera vez en 1945 por Sor Celine Fasenmyer. La existencia del operador $p(n, N)$ implica que, al aplicar ambos lados de la ecuación (2.5) y sumar sobre k , se obtiene una relación de recurrencia para $f(n) = \sum_k F(n, k)$. En efecto, dado que $N^j(F(n, k)) = F(n+j, k)$, al aplicar la linealidad de la suma se concluye que

$$\sum_{j=0}^J a_j(n)f(n+j) = 0.$$

En particular:

- Si $J = 1$, se obtiene una recurrencia de primer orden:

$$a_0(n)f(n) + a_1(n)f(n+1) = 0,$$

de donde se puede despejar explícitamente:

$$f(n) = f(0) \prod_{j=0}^{n-1} \frac{(-1)^n a_0(j)}{a_1(j)}.$$

- Si $J > 1$ y los coeficientes $a_j(n)$ son constantes, se obtiene una recurrencia lineal con coeficientes constantes, la cual puede resolverse mediante los métodos clásicos descritos en la Sección 4.2.
- Si $J > 1$ y cada $a_j(n)$ es un polinomio en n , entonces la recurrencia puede resolverse utilizando el **algoritmo de Petkovšek [Pet]**, también conocido como el **algoritmo Hyper**.

Ejemplo 2.9 A continuación mostramos cómo utilizar el algoritmo de Zeilberger en *Mathematica*[®] para evaluar la suma

$$\sum_{k=0}^n \binom{2k}{k} \binom{2n-2k}{n-k} = 4^n.$$

Esta identidad, conocida actualmente como la **identidad de Hajós** (cf. [Duarte]), admite diversas demostraciones, aunque la mayoría de ellas son combinatoriamente elaboradas. Sin embargo, mediante el algoritmo de Zeilberger es posible obtener una prueba algorítmica de esta igualdad. En el Problema 5 de esta sección se pide calcular la versión alternante de esta suma.

Para aplicar el algoritmo, se comienza por cargar el paquete `fastZeil` y definir el término

$$F(n, k) = \binom{2k}{k} \binom{2n-2k}{n-k}.$$

Luego, al ejecutar el comando `Zb[F[n, k], k, n]`, se obtiene una relación de recurrencia para la suma.

```
In[25]:= F[n_, k_] := Binomial[2k, k] Binomial[2n - 2k, n - k]
```

```
In[26]:= Zb[F[n, k], k, n]
```

```
Out[26]= {4 (1 + n) F[k, n] + (-1 - n)F[k, n + 1] ==  
          Δk[F[k, n] R[k, n]]}
```

Como resultado, se deduce que $F(n, k)$ cumple la identidad:

$$4(n+1)F(n, k) - (n+1)F(n+1, k) = G(n, k+1) - G(n, k),$$

donde el certificado es

$$R(n, k) = \frac{G(n, k)}{F(n, k)} = \frac{2k(2n - 2k + 1)}{n - k + 1}.$$

Este certificado puede visualizarse mediante el comando `show[R]` (con `s` minúscula). Al sumar sobre todos los valores de k , se obtiene:

$$4(n + 1)f(n) - (n + 1)f(n + 1) = 0,$$

lo cual implica que $f(n) = 4f(n - 1)$, y por inducción se concluye que

$$f(n) = 4^n f(0) = 4^n.$$

Esta ecuación también puede resolverse utilizando el algoritmo `Hyper`. Para ello, es necesario descargar^a y cargar el archivo `hyper.m` disponible en la página del autor.

In[27]:= `?Hyper`

Out[27]=

Symbol

Hyper [eqn, y [n]] finds at least one hypergeometric solution of the homogeneous equation eqn over the field of rational numbers Q (provided any such solution exists). Hyper [eqn, y [n], Solutions -> All] finds a generating set (not necessarily linearly independent) for the space of solutions generated by hypergeometric terms over Q. Hyper [eqn, y [n], Quadratics -> True] finds solutions over quadratic extensions of Q. Solutions y[n] are given by their rational representations y [n+1]/y[n].

Warning: The worst -case time complexity of Hyper is exponential in the degrees of the leading and trailing coefficients of eqn.

In[28]:= `Hyper[4 (n + 1) f[n] - (n + 1) f[n + 1] == 0, f[n]]`

Out[28]= `{4}`

El resultado anterior indica que la solución es $f(n) = 4^n$. Nótese que las soluciones que entrega esta implementación están expresadas mediante el cociente $f(n + 1)/f(n)$.

^a<https://www.math.upenn.edu/~wilf/progs.html>

Ejemplo 2.10 El siguiente ejemplo corresponde al Problema 1124 de la revista *The College Mathematics Journal*. Para n un entero positivo, se busca una forma cerrada para la suma

$$\text{Sum}(n) := \sum_{k=0}^{2n-1} \binom{2n-1+k}{k} \binom{2n-1}{k} \frac{(-1)^k}{2^k} \cdot \frac{1}{k+1}.$$

Denotemos por $F(n, k)$ el término general de la suma. Aplicando el algoritmo de

Zeilberger a $F(n, k)$, se obtiene la siguiente relación de recurrencia:

$$(-1 + 2n)F(n, k) + 2(1 + n)F(n + 1, k) = G(n, k + 1) - G(n, k), \quad (2.6)$$

donde el certificado es

$$R(n, k) = \frac{G(n, k)}{F(n, k)} = \frac{2k(1 + k)(-1 - 4n)}{(-k + 2n)(1 - k + 2n)}.$$

Al sumar ambos lados de (2.6) sobre $k \in \mathbb{Z}$ (en la práctica, sobre el soporte finito del sumando), se obtiene la recurrencia para $\text{Sum}(n)$:

$$(2n - 1) \text{Sum}(n) + 2(n + 1) \text{Sum}(n + 1) = 0.$$

Esto implica que $\text{Sum}(n)$ satisface la relación

$$\text{Sum}(n + 1) = -\frac{2n - 1}{2(n + 1)} \text{Sum}(n).$$

Resolviendo esta recurrencia con la condición inicial $\text{Sum}(1) = \frac{1}{2}$, se obtiene:

$$\begin{aligned} \text{Sum}(n) &= (-1)^{n-1} \frac{(2n - 3)(2n - 5) \cdots (1)}{(2n)(2n - 2) \cdots (4)} \text{Sum}(1) \\ &= (-1)^{n-1} \frac{(2n - 3)(2n - 5) \cdots (1)}{(2n)(2n - 2) \cdots (4)(2)} \\ &= \frac{(-1)^{n-1}}{2^{2n}(2n - 1)} \binom{2n}{n}. \end{aligned}$$

Esta misma recurrencia también puede obtenerse utilizando el algoritmo Hyper, como se muestra a continuación:

```
In[29]:= Hyper[(2 n - 1) f[n] + 2 (n + 1) f[n + 1] == 0, f[n]]
```

```
Out[29]= {-1+2n / (2(1+n))}
```

Esto significa que el cociente entre términos consecutivos de la sucesión es

$$\frac{\text{Sum}(n + 1)}{\text{Sum}(n)} = -\frac{2n - 1}{2(n + 1)},$$

lo cual coincide con la relación recursiva previamente obtenida.

Ejemplo 2.11 Consideremos las sumas de potencias de coeficientes binomiales:

$$S_\ell(n) := \sum_{k=0}^n \binom{n}{k}^\ell.$$

Para esta suma se conocen fórmulas cerradas cuando $\ell = 0, 1, 2$. Estudiemos cada caso utilizando el algoritmo de Zeilberger. Consideremos el caso $\ell = 1$

```
In[30]:= F[n_, k_] := Binomial[n, k]
```

```
In[31]:= Zb[F[n, k], k, n]
```

```
Out[31]= {2 F[k, n] - F[k, 1+n] == Δk[F[k, n] R[k, n]]}
```

Esto quiero decir que $2S_1(n) - S_1(n+1) = 0$. De aquí es claro que $S_1(n) = 2^n$. Consideremos el caso $\ell = 2$.

```
In[32]:= F[n_, k_] := Binomial[n, k]^2
```

```
In[33]:= Zb[F[n, k], k, n]
```

```
Out[33]= {-2(1+2n)F[k, n] + (1+n)F[k, 1+n] == Δk[F[k, n] R[k, n]]}
```

Esto quiero decir que $-2(1+2n)S_2(n) + (n+1)S_2(n+1) = 0$. Utilizando el algoritmo Hyper se tiene que

```
In[34]:= Hyper[-2(1 + 2n)f[n] + (1 + n)f[1+n] == 0, f[n]]
```

```
Out[34]= {  $\frac{2(1+2n)}{1+n}$  }
```

Por lo tanto,

$$S_2(n+1) = \frac{2(1+2n)}{1+n} S_2(n).$$

De aquí se puede demostrar que $S_2(n) = \binom{2n}{n}$. Consideremos el caso $\ell = 3$.

```
In[35]:= F[n_, k_] := Binomial[n, k]^3
```

```
In[36]:= Zb[F[n, k], k, n]
```

```
Out[36]= {-8(1+n)^2 F[k, n] + (-16 - 21n - 7n^2)F[k, n+1] +  
(2+n)^2 F[k, n+2] == Δk[F[k, n] R[k, n]]}
```

Esto quiero decir que $S_3(n)$ satisface la recurrencia

$$-8(n+1)^2 S_3(n) + (-16 - 21n - 7n^2) S_3(n+1) + (n+2)^2 S_3(n+2) = 0.$$

Utilizando el algoritmo Hyper se tiene que

```
In[37]:= Hyper[-8(n+1)^2 f[n] + (-16 - 21n - 7n^2) f[n + 1]  
+ (n+2)^2 f[n + 2] == 0, f[n]]
```

```
Out[37]= { }
```

Esto significa que la recurrencia lineal con coeficientes polinomiales no tiene solución hipergeométrica. Por lo tanto, $S_3(n)$ no tiene una fórmula cerrada.

2.4.3. El método de Wilf-Zeilberger

El método de Wilf-Zeilberger, también conocido como **método WZ**, es una herramienta útil para probar identidades combinatorias de la forma:

$$\sum_k F(n, k) = r(n),$$

con $r(n) \neq 0$. Esta identidad es equivalente a la ecuación:

$$\sum_k F(n, k)/r(n) = 1.$$

Si pensamos que $F(n, k)/r(n)$ es el sumando original, entonces el problema se reduce a la igualdad

$$\sum_k F(n, k) = \text{constante}.$$

Sea $f(n) = \sum_k F(n, k)$, debemos demostrar que $f(n)$ es constante para todo n . Una forma es demostrar que $f(n+1) - f(n) = 0$ para todo entero no negativo n . Esto se puede hacer si se encuentra una función $G(n, k)$ tal que:

$$F(n+1, k) - F(n, k) = G(n, k+1) - G(n, k).$$

La pareja de funciones (F, G) se denomina la **pareja WZ**. Con este análisis en mente se tiene el algoritmo de la Figura 2.25.

Entrada Un término hipergeométrico $F(n, k)$ respecto a n y k .

Salida Certificado $R(n, k) := G(n, k)/F(n, k)$.

- 1: Definimos $D(k) := F(n+1, k) - F(n, k)$.
- 2: Aplicamos el algoritmo de Gosper a $D(k)$. Si es exitoso encontramos un término $g(k)$ tal que

$$D(k) = g(k+1) - g(k).$$

Como $g(k)$ también depende del parámetro n , entonces tenemos que $g(k) := G(n, k)$.

- 3: $R(n, k) := G(n, k)/F(n, k)$ es racional y es el **certificado WZ** para la identidad

$$\sum_k F(n, k) = \text{constante}.$$

Figura 2.25: Algoritmo WZ.

Vamos a utilizar el código elaborado por Doron Zeilberger para encontrar el certificado WZ. Para ello debemos descargar³ y correr el archivo gosper.m de la página web del autor.

³<https://www.math.upenn.edu/~wilf/progs.html>

Ejemplo 2.12 A continuación mostramos como utilizar el método WZ para demostrar la identidad:

$$\sum_{k=0}^n \binom{n+k}{n} 2^{-k} = 2^n.$$

Para más información sobre esta suma ver la referencia [Ah]. Observe primero que esta suma es equivalente a la ecuación:

$$f(n) := \sum_{k=0}^n \binom{n+k}{n} 2^{-(k+n)} = 1.$$

Así que debemos definir $F(n, k) := \binom{n+k}{n} 2^{-(k+n)}$.

```
In[38]:= << gosper'
```

N.B.: Besides GosperSum and GosperFunction, this

package also contains FactorialSimplify (alias FS), and WZ.

```
In[39]:= ?WZ
```

```
Out[39]=
```

Symbol
WZ[F, n, k, qList: {}] returns a rational function R (n,k)
such that F (n,k) and G (n,k) := R(n,k) F(n,k) are a WZ pair:
F(n+1,k) - F(n,k) = G(n,k+1) - G(n,k).

```
In[40]:= F[n_, k_] := Binomial[n + k, n] 2^(-k - n)
```

```
In[41]:= WZ[F[n, k], n, k]
```

```
Out[41]= -(k/(1 + n))
```

```
In[42]:= R[n_, k_] := -(k/(1 + n))
```

```
In[43]:= G[n_, k_] := R[n, k]*F[n, k]
```

```
In[44]:= FullSimplify[F[n + 1, k] - F[n, k] - (G[n, k + 1] - G[n, k])]
```

```
Out[44]= 0
```

En este caso el certificado es

$$R(n, k) = -\frac{k}{n+1}.$$

Sumando sobre todo k , se concluye que

$$f(n+1) - f(n) = 0.$$

Así que $f(n)$ es constante e igual a $f(0) = 1$. Quedando demostrado.